AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 06.10.2026

Lab 01

Podstawy Pythona: Hello, World! i BMI

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **co dzieje się z kodem** od wpisania do wykonania
- Skonfigurować środowisko: **Python, VS Code, Git, GitHub**
- Napisać i uruchomić pierwsze skrypty: zmienne, `input()`, `print()`, f-stringi
- Oddać pracę przez Git i sprawdzić ją testami automatycznymi
- Efekty z sylabusa: **U1** (skrypty w Pythonie), **KS1** (samodzielna praca)



CZĘŚĆ I

Teoria: od algorytmu do działającego programu

Algorytm

> **Definicja.** Algorytm to skończony ciąg jednoznacznie określonych kroków, który przekształca dane wejściowe w wynik.

Pięć cech algorytmu według D. Knutha (*The Art of Computer Programming*, t. 1):

CECHA	ZNACZENIE
skończoność	kończy się po skończonej liczbie kroków
określoność	każdy krok jest jednoznaczny
wejście	ma zero lub więcej danych wejściowych
wyjście	daje co najmniej jeden wynik
efektywność	każdy krok da się faktycznie wykonać

Nazwa pochodzi od perskiego matematyka al-Chwarizmiego (IX w.), łac. *Algoritmi de numero Indorum*.

Od algorytmu do programu

- **Język programowania** to formalny zapis algorytmu zrozumiały dla maszyny i człowieka
- **Składnia** mówi, jak zapis ma wyglądać; **semantyka** – co oznacza
  - `print("Hi"` → błąd składni (`SyntaxError`), program nie ruszy
  - `print(10 / 0)` → składnia poprawna, błąd w trakcie działania (`ZeroDivisionError`)
- Poziomy języków:

POZIOM	PRZYKŁAD	BLISKO…
kod maszynowy	`10110000 01100001`	procesora
asembler	`mov al, 61h`	procesora
wysoki poziom	`x = 97`	człowieka

Kompilacja czy interpretacja?

- **Kompilator** tłumaczy cały program na kod maszynowy *przed* uruchomieniem (C, Rust, Go)
- **Interpreter** wykonuje program *w trakcie* czytania (powłoka `bash`)
- **Python (CPython) łączy oba podejścia:**

```
kod źródłowy (.py) ──kompilacja──► kod bajtowy ──► maszyna wirtualna Pythona (PVM)
```

- Kod bajtowy importowanych modułów trafia do `__pycache__/*.pyc`, żeby nie kompilować go ponownie
- Dlatego mówimy, że Python jest językiem **interpretowanym**: kompilacja jest ukryta i automatyczna

Zajrzyj pod maskę: kod bajtowy

```
import dis
dis.dis(compile('print("Hello, World!")', '<s>', 'exec'))
```

```
LOAD_NAME     0 (print)
PUSH_NULL
LOAD_CONST    0 ('Hello, World!')
CALL          1
POP_TOP
RETURN_CONST  1 (None)
```

- Jedna linia Pythona to kilka prostych instrukcji maszyny wirtualnej
- Maszyna wirtualna jest **stosowa**: wkłada wartości na stos i wywołuje na nich operacje

Wynik z Pythona 3.13; nazwy instrukcji różnią się między wersjami.

Model danych: wszystko jest obiektem

> **Dokumentacja Pythona:** każdy obiekt ma **tożsamość**, **typ** i **wartość**.

```
x = 42
id(x)       # tożsamość – stała przez całe życie obiektu
type(x)     # <class 'int'>
x           # 42 – wartość
```

- **Zmienna to nazwa (etykieta)** przypięta do obiektu, a nie „pudełko" na wartość
- `y = x` nie kopiuje liczby, tylko przypina drugą etykietę do tego samego obiektu
- Nazwy zmiennych: `snake_case`, litery, cyfry i `_`, nie mogą zaczynać się cyfrą

docs.python.org/3/reference/datamodel.html

Typowanie: dynamiczne i silne

- **Dynamiczne:** typ ma *wartość*, nie zmienna. Typ sprawdzany jest w trakcie działania
- **Silne:** Python nie zamienia typów „po cichu” przy niepasujących operacjach

```
x = 5          # int
x = "pięć"     # teraz str – wolno
"5" + 5        # TypeError: can only concatenate str (not "int") to str
```

	SILNE	SŁABE
dynamiczne	Python	JavaScript (`"5" + 5` → `"55"`)
statyczne	Java	C

Wejście i wyjście: strumienie

Każdy program dostaje od systemu trzy **standardowe strumienie**: `stdin` (wejście), `stdout` (wyjście) i `stderr` (błędy).

- `input(prompt)` wypisuje zachętę, **czyta jedną linię z `stdin`** i zwraca ją jako `str` (bez znaku nowej linii)
- `print(*obiekty, sep=" ", end="\n")` pisze do `stdout`

```
print("a", "b", "c", sep="-")     # a-b-c
print("bez nowej linii", end="")
```

- **Nasze testy korzystają dokładnie z tego mechanizmu:** uruchamiają Twój skrypt, podają dane na `stdin` i czytają `stdout`

f-stringi i mini-język formatowania

Wprowadzone w **Pythonie 3.6** (PEP 498). Wyrażenie w `{}` jest obliczane i wstawiane do tekstu.

ZAPIS	WYNIK	ZNACZENIE
`f"{1234567.891:,.2f}"`	`1,234,567.89`	separator tysięcy, 2 miejsca
`f"{0.4567:.1%}"`	`45.7%`	procent
`f"{42:08b}"`	`00101010`	binarnie, 8 znaków, zera z przodu
`f"{'x':>5}"`	`x`	wyrównanie do prawej
`f"{pi=:.2f}"`	`pi=3.14`	tryb diagnostyczny (Python 3.8+)

Format: `{wartość:[wypełnienie][wyrównanie][szerokość][,][.precyzja][typ]}`

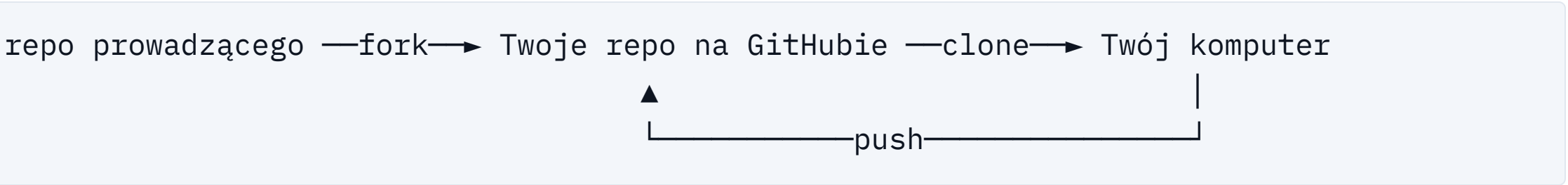
Styl kodu: PEP 8 i Zen Pythona

> *„Kod czyta się znacznie częściej, niż się go pisze."* – PEP 8 (G. van Rossum, B. Warsaw, N. Coghlan, 2001)

- **PEP** (*Python Enhancement Proposal*) to dokument, w którym społeczność proponuje i opisuje zmiany w języku
- Najważniejsze zasady PEP 8 na start:
  - wcięcie: **4 spacje**
  - nazwy: `zmienne_i_funkcje`, `STALE`, `NazwyKlas`
  - spacje wokół `=` i operatorów: `bmi = weight / height ** 2`
- `import this` → **Zen Pythona** (PEP 20): *„Czytelność się liczy."*

Git: kontrola wersji

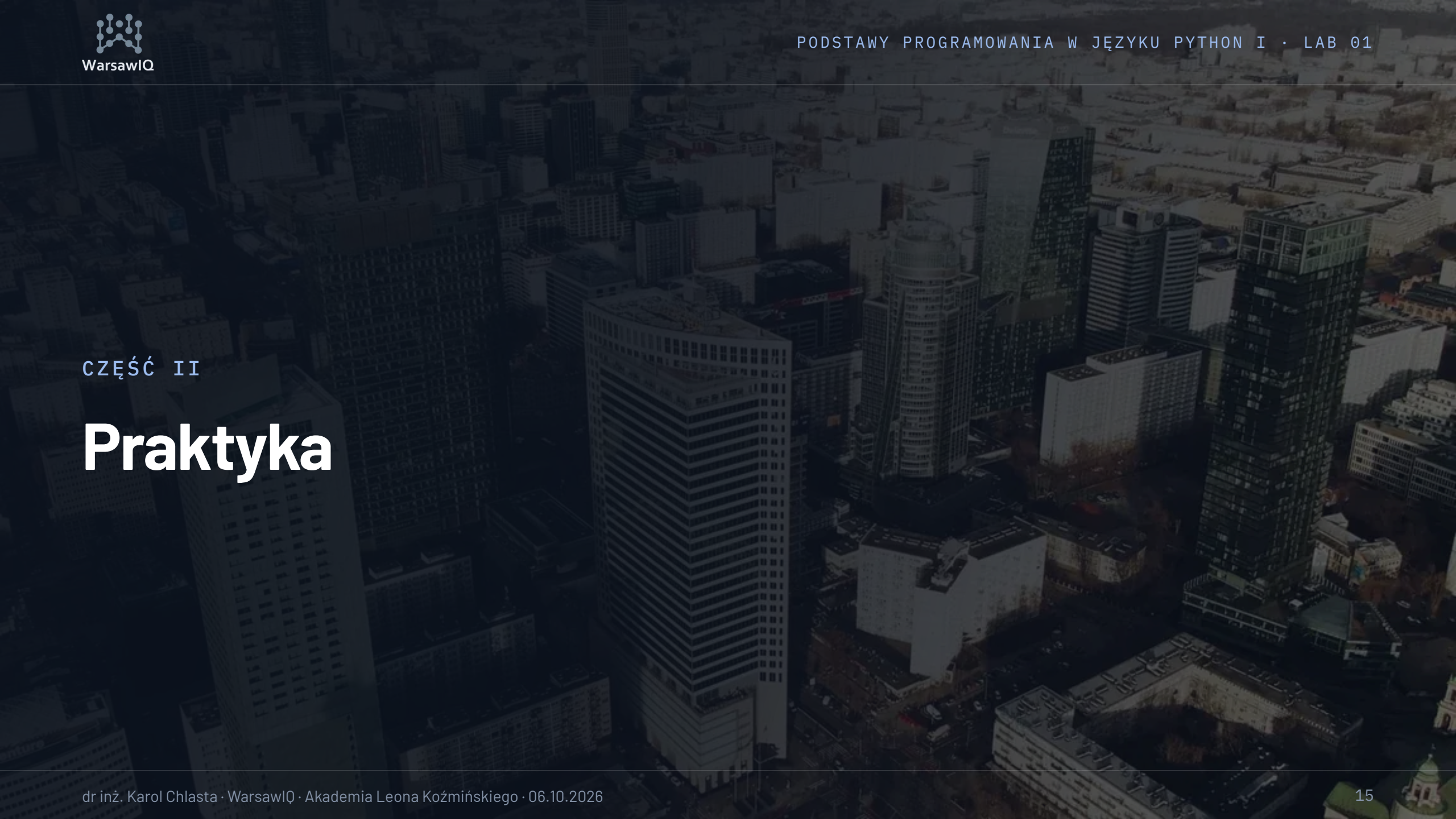
- **Git** to rozproszony system kontroli wersji, napisany przez Linusa Torvaldsa w 2005 r. na potrzeby jądra Linuksa
- **Commit** to migawka (*snapshot*) całego projektu z opisem, autorem i czasem
- Na zajęciach pracujemy tak:



- `git add` → wybierz zmiany, `git commit` → zapisz migawkę, `git push` → wyślij na GitHub
- **GitHub Actions** uruchamia testy po każdym pushu i zapisuje czas oddania

Testy automatyczne

- **Test** to program, który sprawdza inny program: podaje dane i porównuje wynik z oczekiwanym
- Moduł `unittest` jest w bibliotece standardowej Pythona
- Jak działa test do `zad3_bmi.py`:
  i. uruchamia Twój skrypt jako osobny proces,
  ii. wpisuje na `stdin`: `70` i `1.75`,
  iii. szuka w `stdout` liczby `22.86`.
- `OK` → zadanie zaliczone · `FAIL` → test pokazuje, czego oczekiwał
- Testy nie zastępują myślenia: przejście testów nie dowodzi, że program jest poprawny **dla każdych danych**



CZĘŚĆ II

Praktyka

KONFIGURACJA KROK PO KROKU

Środowisko programisty od zera – zrób to razem z prowadzącym

Co instalujemy i po co?

NARZĘDZIE	DO CZEGO SŁUŻY	ANALOGIA
Python	wykonuje Twój kod	silnik
VS Code	edytor, w którym piszesz kod	warsztat
Git	zapisuje historię zmian w kodzie	„zapis gry”
GitHub	przechowuje kod w chmurze, uruchamia testy	dysk w chmurze + sprawdzarka

Repozytorium kursu: **github.com/KarolChlasta/KU-PyDev**

Pełna instrukcja: plik **SETUP.md** w repozytorium

Słowniczek na start

- **Terminal** (konsola) – okno, w którym wpisujesz polecenia zamiast klikać
- **Folder (katalog)** – to samo; w terminalu poruszasz się po folderach
- **Repozytorium (repo)** – folder projektu z historią zmian
- **Fork** – Twoja własna kopia cudzego repo na GitHubie
- **Clone** – pobranie repo z GitHuba na komputer
- **Commit** – zapisany „punkt kontrolny” z opisem zmian
- **Push** – wysłanie commitów z komputera na GitHub

Krok 1: Python

1. Wejdź na **python.org/downloads** → pobierz najnowszą wersję (3.12 lub nowszą)
2. **Windows:** w pierwszym oknie instalatora zaznacz ✅ **„Add python.exe to PATH"** → *Install Now*
3. **macOS:** uruchom pobrany instalator `.pkg` i klikaj *Dalej*
4. **Linux:** Python zwykle już jest; brakujące pakiety: `sudo apt install python3 python3-venv`

Sprawdź w terminalu:

```
python --version     # Windows  → Python 3.13.x
python3 --version    # macOS / Linux
```

Windows: jeśli „python" otwiera Microsoft Store, zainstaluj ponownie z zaznaczonym PATH albo użyj polecenia py.

Krok 2: VS Code

1. Pobierz z **code.visualstudio.com** i zainstaluj (opcje domyślne)
2. Otwórz VS Code → ikona **Extensions** po lewej (`Ctrl+Shift+X`)
3. Wyszukaj i zainstaluj **Python** (wydawca: Microsoft)
4. Otwórz wbudowany terminal: menu **Terminal** → **New Terminal** (`Ctrl+\``)

> Od teraz wszystkie polecenia wpisujesz w terminalu **wewnątrz VS Code** – na dole okna.

Terminal bez stresu

POLECENIE	CO ROBI
`pwd`	pokazuje, w jakim folderze jesteś (*Windows PowerShell też to rozumie*)
`ls`	wypisuje zawartość folderu (Windows: `ls` lub `dir`)
`cd nazwa_folderu`	wchodzi do folderu
`cd ..`	wraca folder wyżej
`python plik.py`	uruchamia program

- **Tab** podpowiada nazwy plików – nie przepisuj ich ręcznie
- **Strzałka** ↑ przywołuje poprzednie polecenie
- Błąd w terminalu to nie katastrofa – **przeczytaj ostatnią linijkę**, zwykle mówi, co jest nie tak

Krok 3: Git

1. **Windows:** pobierz z **git-scm.com** → instaluj z opcjami domyślnymi
   macOS: wpisz w terminalu `git --version` → system zaproponuje instalację
   Linux: `sudo apt install git`
2. **Zamknij i otwórz ponownie VS Code**, żeby terminal „zobaczył" Gita
3. Przedstaw się Gitowi (raz na komputerze):

```
git config --global user.name "Imię Nazwisko"
git config --global user.email "twoj@email"
git --version
```

Krok 4: GitHub i fork

1. Załóż konto na **github.com** (zapamiętaj login – podasz go prowadzącemu)
2. Wejdź na **github.com/KarolChlasta/KU-PyDev** → przycisk **Fork** → **Create fork**
3. W **swoim** forku otwórz zakładkę **Actions** → kliknij
   „I understand my workflows, go ahead and enable them"

> ⚠️ Bez kroku 3 GitHub nie zapisze czasu oddania – **nie dostaniesz punktów za aktywność**.

Polecane: **education.github.com** – darmowy GitHub Copilot dla studentów (przyda się na labach 07, 13, 14).

Krok 5: pobierz repo na komputer (clone)

Sposób A – w VS Code (najprostszy):

1. `Ctrl+Shift+P` → wpisz **Git: Clone** → Enter
2. Wklej adres **swojego** forka: `https://github.com/TWOJ_LOGIN/KU-PyDev.git`
3. Wybierz folder (np. *Dokumenty*) → **Open**

Sposób B – w terminalu:

```
git clone https://github.com/TWOJ_LOGIN/KU-PyDev.git
cd KU-PyDev
code .
```

Krok 6: uruchom pierwszy program i testy

1. Po lewej (Explorer) otwórz `lab01_podstawy/zad1_hello.py`
2. Wpisz pod komentarzem: `print("Hello, World!")` → zapisz `Ctrl+S`
3. Uruchom przyciskiem ▶ **(Run Python File)** w prawym górnym rogu
4. Sprawdź testami w terminalu:

```
cd lab01_podstawy
python -m unittest discover -s tests -v
```

`OK` = działa ✅ · `FAIL` = przeczytaj, czego test oczekiwał ❌

Krok 7: pierwszy commit i push

W VS Code – panel Source Control (`Ctrl+Shift+G`):

1. Przy zmienionym pliku kliknij **+** (*Stage*)
2. Wpisz opis, np. `Lab 01` → **Commit**
3. Kliknij **Sync Changes** / **Push** → przy pierwszym razie zaloguj się do GitHuba w przeglądarce

Albo w terminalu:

```
git add .
git commit -m "Lab 01"
git push
```

Na GitHubie w zakładce **Actions** pojawi się ✅ lub ❌ – to Twoje potwierdzenie oddania.

Gotowe? Lista kontrolna ✅

- [ ] `python --version` pokazuje 3.12 lub nowszy
- [ ] VS Code ma rozszerzenie **Python**
- [ ] `git --version` działa, `user.name` i `user.email` ustawione
- [ ] Mam **fork** i włączone **Actions**
- [ ] Repo sklonowane i otwarte w VS Code
- [ ] `zad1_hello.py` działa, testy uruchomione
- [ ] Pierwszy **push** widoczny na GitHubie

Coś nie działa? Podnieś rękę – albo zajrzyj do **SETUP.md → Najczęstsze problemy**.

Pierwszy program

```
print("Hello, World!")
```

```
name = input("Jak masz na imię? ")
print(f"Cześć, {name}!")
```

`input()` **zawsze** zwraca tekst → do obliczeń użyj `int(...)` lub `float(...)`

BMI w trzech liniach

```
weight = float(input("Masa [kg]: "))
height = float(input("Wzrost [m]: "))
print(f"BMI = {weight / height ** 2:.2f}")
```

- `**` ma wyższy priorytet niż `/`, więc liczymy `weight / (height ** 2)`
- `:.2f` – 2 miejsca po przecinku (tylko wyświetlanie, wartość się nie zmienia)
- liczby dziesiętne wpisujemy z **kropką**: `1.75` (`float("1,75")` → `ValueError`)

Ciekawostki

- Nazwa **Python** pochodzi od serialu *Latający Cyrk Monty Pythona*, nie od węża
- Guido van Rossum zaczął pisać Pythona w **grudniu 1989** jako „projekt na święta"; pierwsza publiczna wersja (0.9.0) – **luty 1991**
- **Python 3.0** (grudzień 2008) celowo zerwał zgodność z Pythonem 2; wsparcie Pythona 2 skończyło się **1 stycznia 2020**
- Wpisz w Pythonie:
  - `import this` → *Zen Pythona* (Tim Peters)
  - `import antigravity` → komiks xkcd o Pythonie

Ciekawostki: BMI

- Podstawy wskaźnika opracował belgijski astronom i statystyk **Adolphe Quetelet** w latach **1830–1850** – stąd dawna nazwa „wskaźnik Queteleta”
- Nazwę *Body Mass Index* wprowadził **Ancel Keys** w artykule z 1972 r.
- BMI nie odróżnia mięśni od tłuszczu – kulturysta może mieć „nadwagę”
- To dobry przykład ogólnej zasady: **model upraszcza rzeczywistość**, a program liczy dokładnie to, co mu każesz

Dzisiejsze zadania

	PLIK	ZADANIE
★	—	Konfiguracja: fork, clone, pierwszy commit
★	`zad1_hello.py`	Hello, World!
★	`zad2_wizytowka.py`	Wizytówka z f-stringiem
★	`zad3_bmi.py`	Kalkulator BMI
☆	`zad4_bmi_kategoria.py`	Kategoria BMI (zapowiedź `if`)

Sprawdzenie i oddanie

```
cd lab01_podstawy
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 01"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- Liczy się czas na GitHubie, nie data commita. Po zajęciach – **0 pkt**, bez wyjątków
- Zaliczenie: **Kolokwium I (50) + Kolokwium II (50) + bonus do 20** (obecność + aktywność)

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 1–2
- Oficjalny samouczek: *docs.python.org/3/tutorial* – rozdz. 1–3
- PEP 8 – *Style Guide for Python Code*: *peps.python.org/pep-0008*
- S. Chacon, B. Straub, *Pro Git* (bezpłatnie: *git-scm.com/book/pl*) – rozdz. 1–2

Następny lab (13.10): zmienne, typy danych i operacje arytmetyczne